The Fifty Overs match engine

How one delivery becomes cricket, where a broad 0–100 player world breaks it, and what has been measured, fixed and left undone. Phase B1.

The finding, in one table

Three deliveries, computed by the shipped engine. Mid-innings, balanced pitch, settled batsman, uniform skills. Probabilities in per cent.

contest	dot	1	2	4	6	wicket
85 bat v 85 bowl	37.73	37.80	6.88	7.52	1.42	1.33
95 bat v 85 bowl	37.65	37.85	6.88	7.54	1.42	1.33
85 bat v 95 bowl	37.72	37.80	6.88	7.54	1.42	1.33

These are the same delivery. A generational batsman facing a good bowler receives an identical ball to a good batsman facing a great one. The differences are in the third decimal place — noise, not cricket.

The engine cannot tell, at elite level, which of the two men is better. That single fact explains the dead top end we have been chasing, and it is the reason a world running from amateur to legend cannot be built on the engine as it stands. Everything else in this report is either the mechanism behind it or the evidence for it.

1. What happens when a ball is bowled

The whole simulation lives in `engine/src/00-core.js`. One delivery runs through six stages.

#	function	line	what it does
1	stepBall()	1168	Reads the two men, the over, the field, fatigue, intent, keeper, talents and match state
2	ballDist(…)	233	Builds a log-odds score for every possible outcome, then softmaxes them into probabilities
3	foTuneDist(…)	~562	A multiplicative tuning layer applied after the model
4	outcome draw	~1210	ONE uniform random number walks the cumulative distribution
5	fielding	after	Catch or drop, run-out, boundary saved — further draws
6	scoreboard	tail	Runs, wickets, balls, strike, over, innings state

There is exactly one probability vector per ball. The model is not staged — it does not decide “wicket or not” and then “how many runs”. Dot, 1, 2, 3, 4, 6, five kinds of dismissal and four kinds of extra all compete in a single softmax, and one random draw picks the winner. This is a sound design and is not the problem.

The outcome buckets

```
dot   1   2   3   4   6
wC (caught)   wB (bowled)   wLBW   wRO (run out)   wST (stumped)
wide   noball   bye   legbye
```

2. Where player skill enters — and the architectural flaw

Skill reaches the ball through exactly three channels.

Channel 1 and 2 — absolute quality, independently softened

```
bs = 10 · tanh( (bat.bat    - 56.6) / 10 )      the batsman
wt = 10 · tanh( (bowl.threat - 68.1) / 10 )      the bowler
```

Each man is measured against a fixed global average and the result is squashed by `tanh` with a ceiling of 10. The two numbers are then *subtracted from each other in the log-odds* — but each has already been flattened on its own, so the comparison happens after the information has gone.

Channel 3 — the difference, which is switched off most of the time

```
mmRaw  = (threat - 68.1) - (bat - 56.6) - (-10)
mmOver = max( 0, |mmRaw| - 15 )          <- the dead band
mm     = sign · 2.0 · tanh( (mmOver²/300) / 2.0 )
```

This is the only term in the engine that genuinely compares the two cricketers, and it is exactly zero until they are roughly 25 skill points apart (`mismatch_free = 15` combined with `mismatch_pivot = -10`).

Why the three deliveries came out identical

For a 95 batsman against an 85 bowler:

```
mmRaw  = (85 - 68.1) - (95 - 56.6) + 10  =  -11.5
|-11.5| < 15   ->   mmOver = 0   ->   mm = 0        (no mismatch at all)

bs(95) = 10·tanh(38.4/10) = 9.99
bs(85) = 10·tanh(28.4/10) = 9.94
difference 0.05, multiplied by skill_bat 0.011  =  0.0006 on the logit
```

Ten points of world-class talent are worth six ten-thousandths of a log-odds unit. That is the dead top end, proven from the code rather than inferred from win rates.

A fourth, quieter channel

```
stdRaw = (((bat - 56.6) + (threat - 68.1)) / 2 + 8) / 10
std    = 1.5 · tanh( stdRaw / 1.5 )
```

`std` reads the **sum** of the two men, not their difference — it represents "what standard of cricket is this", and it correctly makes better cricket score more and lose fewer wickets. It is not a comparison.

3. How much information survives the transforms

Pure arithmetic. What a ten-point improvement in raw skill is worth to the engine, at different points on the scale, for several values of the softening constant (`skill_soft`; shipped value is 10).

raw step	soft=10	soft=15	soft=20	soft=30	soft=50	linear
20 -> 30	0.084	0.614	1.612	3.897	6.869	10.000
25 -> 35	0.227	1.157	2.508	4.986	7.622	10.000
45 -> 55	6.624	8.139	8.857	9.456	9.797	10.000
55 -> 65	8.445	9.214	9.535	9.786	9.921	10.000
70 -> 80	1.099	3.036	4.786	7.007	8.741	10.000
80 -> 90	0.159	0.924	2.146	4.576	7.354	10.000
85 -> 95	0.059	0.487	1.366	3.546	6.594	10.000

*At the shipped setting a ten-point step is worth **8.4** points of engine signal in the middle of the range and **0.06** at the top — a factor of 143. The usable window is roughly raw 45 to 70, twenty-five points of a hundred-point scale.*

4. The causal bisect

Six configurations of the engine, 13 matchups each, 300 matches per cell, sides swapped and played on neutral ground. Each run turns off or widens exactly one term. This is the repo's own method for proving a term responsible — reading a formula and reasoning about it produced two confident wrong diagnoses in this investigation, and only the bisect corrected them.

configuration	25->35	50->60	70->80	85->95	50v50 runs	85v85 runs	85v85 wkts
shipped	86.7%	73.7%	61.7%	52.3%	300	318	3.67
skill_soft 15	88.3%	75.3%	64.0%	56.3%	310	329	3.36
skill_soft 30	87.7%	77.0%	63.7%	60.0%	329	357	3.05
skill_soft 100	93.0%	84.0%	67.7%	55.7%	359	417	1.92
mismatch off	80.0%	70.7%	62.0%	52.3%	313	320	3.60
standard off	85.3%	69.7%	62.7%	60.7%	187	185	8.12

What it establishes

- **The mismatch term owns the low-end collapse, and owns it alone.** Disabled, a side of mean skill 25 facing one of 70 goes from 65.6 all out to 234.8, and from 9.95 wickets to 7.77. Nothing else moved that number — tripling the softening made it worse (54.5) and zeroing the standard terms made it worse still (31.1).
- **The standard terms own the scoring level.** Zeroed, every grade of cricket collapses to about 180 all out with an all-out rate over half — the best cricket in the world included. They are load-bearing.
- **Widening the softening is not a fix.** It buys about eight points of top-end resolution for forty runs of score inflation, does nothing whatever for the low-end cliff, and at extreme settings stops equal sides being neutral (25 v 25 comes out 57.7%).
- **A hypothesis of mine was refuted.** I had argued from algebra that the softening caused the low-end cliff. It does not: 25->35 measures 86.7%, 88.3%, 87.7% and 93.0% as the constant triples — flat, then worse.

5. What has been changed, and what it bought

Two contained changes to `ballDist`, both written in the engine's own `tanh` idiom, both leaving ordinary cricket where it was calibrated. Committed as `a115e30` on the working branch; `main` is untouched.

Bounding the mismatch

```
before   mm = sign · mmOver² / 300                    (unbounded)
after    mm = sign · 2.0 · tanh( (mmOver²/300) / 2.0 )
```

The old term reached 6.3 against an amateur, which the wicket coefficient turned into **+2.9 on a logit whose base is about -2.7** — a coin-toss wicket every ball. Below the cap the curve is virtually unchanged, so every mismatch a real league produces passes through as before.

Bounding the standard

```
before   std = stdRaw                                 (unbounded, linear)
after    std = 1.5 · tanh( stdRaw / 1.5 )
```

At 85 against 85 the old term reached 3.07 and took 0.77 off the wicket logit on its own, so an innings arrived at roughly 3.7 wickets and 318 runs *before anybody's relative advantage was considered*. An innings cannot lose much under three wickets, so a better batsman had nowhere to put his extra quality.

measure	before	after
25 v 70 — weak side runs	65.6	117.4
25 v 50 — weak side runs	119	160.9
25 v 35 — stronger side wins	86.7%	84.8%
85 v 95 — stronger side wins	52.3%	56.8%
50 v 50 — first innings	300	255
70 v 70 — first innings	320	246
85 v 85 — first innings	318	239
85 v 85 — wickets lost	3.67	6.48

The tails survived, which was the risk

A cap on a skill term can flatten a distribution while leaving its mean exactly where it was, so the mean is the one statistic that cannot detect the damage. Measured at 700 matches a cell, the bounded engine still produces cricket at both extremes, naturally and at believable rarity.

contest	min	5th pct	median	95th pct	max	under 50	over 400
25 v 70	9	79	220	423	455	1.6%	14.1%
25 v 50	21	110	271	390	427	0.6%	2.1%
50 v 50	60	188	278	326	359	0%	0%
70 v 70	7	195	264	302	334	0.1%	0%
85 v 85	101	195	254	292	340	0%	0%

A nine-all-out and a 455 both remain reachable by ordinary sequences of deliveries, and neither is forced by anything. No score caps, floors, rubber-banding or innings-total normalisation exist anywhere in the engine — audited specifically.

6. Every nonlinearity in the delivery model

element	formula	what it does	verdict
SOFT	10·tanh(v/10)	Saturates 25 points from the mean, in BOTH directions	Dangerous
mismatch dead band	max(0,|gap|-15)	Zero response inside a 30-point window	Dangerous
mismatch curvature	over²/300	Small gaps nothing, large gaps explode	Suspicious
mismatch cap (new)	2.0·tanh(x/2)	Bounds the low-end cliff	Healthy
standard cap (new)	1.5·tanh(x/1.5)	Restores wicket headroom; removed the last channel separating 85 from 95	Suspicious
softmax	exp(lo)/SUMexp	Log-odds to probability compression near 0 and 1	Healthy (inherent)
new ball / grip	exp(-over/8), logistic	Phase shape over the innings	Healthy
field average	clamp to [-1.2, 1.4]	Hard clamp on team fielding	Suspicious
foTuneDist	multiplicative	Post-model tuning layer	Unknown — not audited

7. Hidden scale assumptions

The engine was fitted around a population that no longer describes the world we want to build. Every skill term is centred on a hard-coded average:

```
mean_bat 56.6    mean_thr 68.1    mean_pow 47    mean_rot 57.7    mean_ctl 64.6
mismatch_pivot -10    mismatch_free 15    standard_pivot -8    skill_soft 10
```

`mean_thr = 68.1` carries the code comment “the best bowler alive has 68” — so the bowling term is *always* negative in the shipped world. Together these constants define a usable band of roughly twenty-five skill points, which is precisely why the entire cricket world, from the bottom of the second division to the best national side, has had to be squeezed into raw skill 20–35.

8. What is left to do

One architectural change, not a rewrite. The pipeline, the single-softmax outcome selection, the correlated match state that produces genuine fat tails, the conditions and the talents are all sound and worth keeping.

But the engine is fundamentally *absolute quality plus absolute quality*, with one relative term that is inert across a thirty-point window. No amount of parameter tuning fixes that, because at elite level there is nothing in the arithmetic that reads “this batsman is better than this bowler”.

The fix is to make the batsman-versus-bowler **difference** a first-class, continuous input: either remove the dead band and soften the mismatch smoothly from zero, or re-express the two independently-softened absolute terms as one softened differential. The probability table on the first page is direct evidence for this rather than a hypothesis.

Honest remaining state

- The low end is improved but not solved: a ten-point gap is still worth about 85% at the bottom against 57% at the top. Measurement shows this is driven by the low-scoring, high-wicket environment down there rather than by any skill term — a brittle innings converts a small per-ball edge into a large win-probability edge.
- The standard cap should be re-examined once the difference term is fixed. It may prove unnecessary, or want a higher ceiling.
- Three validations remain unrun: the batting-versus-bowling grid, the individual career curves at 70/80/85/90/95, and the production calibration retest. Each is a single command against the committed harness.
- The extras buckets do not scale with standard, so at very low totals they become a large share of the score. Plausible low-end distortion; unmeasured.

9. Tooling, evidence and commits

commit	what it is
3d20a90	strength-response harness — measures the engine across the full range
78486d4	bisect handle — turns one tuning term off at a time inside the test VM
775084a	causal evidence — the six bisect configurations and their verdict
a115e30	the engine fix — bounded mismatch and bounded standard, plus 8 regression tests
53b6334	tail evidence — percentile distributions proving extremes survived
eda9412	the arithmetic showing why sweeping the mismatch cannot smooth the low end

Everything lives on the working branch. `main` *has not been touched, the frozen golden master has not been re-blessed, and no part of the player world — ratings, wages, squads, national teams — has been modified. Raw output is under* `docs/b1-evidence/`.

Engine test suite: ***370 of 371 pass****, plus 8 new regression tests. The single failure is the bit-for-bit golden-master replay, which cannot pass after an authorised engine change and must not be edited to — that test existing is how we know the engine moved.*